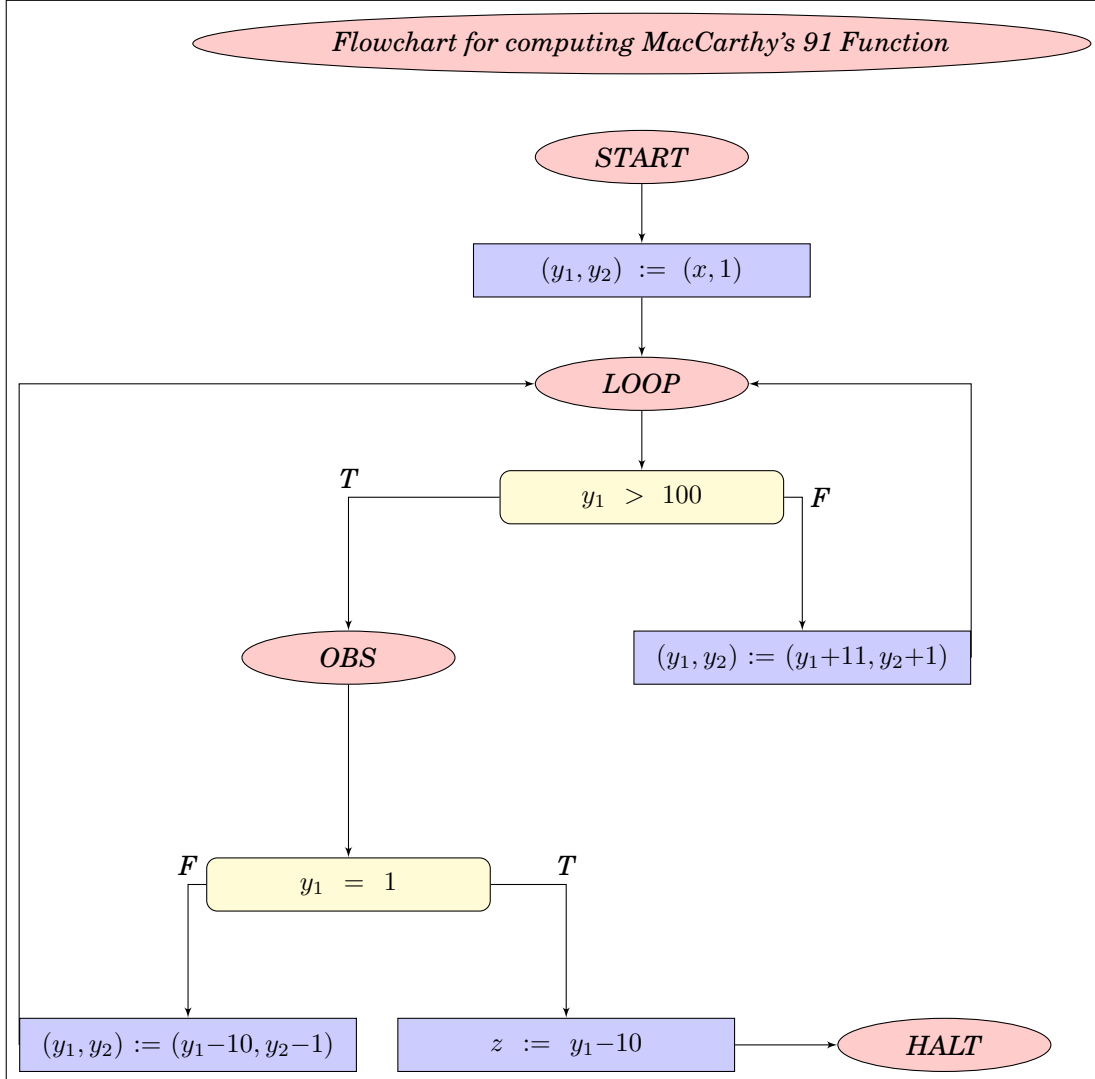


<

Exercice 1 Soit l'organigramme suivant proposé par Z. Manna dans son ouvrage *Mathematical Theory of Computation*.



Question 1.1 Construire un module TLA⁺ modélisant les différents pas de calcul.

Question 1.2 Evaluer l'algorithme en posant des questions de sûreté suivantes :

1. l'algorithme est partiellement correct.
2. l'algorithme n'a pas d'erreurs à l'exécution.

◇ **Solution de l'exercice 1**

Listing 1 – Module TLA+

```
1 ----- MODULE appex2_2
```

```

2  (* Calcul de la fonction de MacCarthy *)
3  EXTENDS Naturals, TLC, Integers
4  -----
5  CONSTANTS x, max, u
6  min == -max
7  -----
8  VARIABLES  y1, y2, z, pc
9  -----
10 BF(Y) == Y#u => Y \in min..max
11
12 ASSUME  BF(x)
13
14 -----
15
16 start == pc="START" /\ y1'=x /\ y2'=1 /\ pc'="LOOP"  /\ UNCHANGED <<z>>
17
18 case1 ==
19     /\ pc="LOOP" /\ y1 \leq 100
20     /\ y1'=y1+1 /\ y2'=y2+1
21     /\ UNCHANGED <<z, pc>>
22
23 case2 ==
24     /\ pc="LOOP" /\ y1 > 100
25     /\ pc'="OBS"
26     /\ UNCHANGED <<z, y1, y2>>
27
28 case21 ==
29     /\ pc="OBS" /\ y2#1
30     /\ y1'=y1-10 /\ y2'=y2-1
31     /\ pc'="LOOP"
32     /\ UNCHANGED <<z>>
33
34 case22 ==
35     /\ pc="OBS" /\ y2=1
36     /\ z'=y1-10 /\ pc'="HALT"
37     /\ UNCHANGED <<y1, y2>>
38     -----
39
40 ePrint == pc="HALT" /\ PrintT(z) /\ UNCHANGED <<y1, y2, z, pc>>
41
42 -----
43 Next == start \/ case1 \/ case2 \/ case21 \/ case22 \/ UNCHANGED <<y1, y2, z, pc
44     >> \/ ePrint
45 init1 == y1 \in Int /\ y2 \in Int /\ z \in Int /\ pc = "START"
46 Init  == y1 = u /\ y2 = u /\ z = u /\ pc = "START"
47
48 -----
49 Q1 == pc#"HALT" (* c prned la valeur HALT *)
50 Qpartialcorrectness == pc="HALT" => z=IF x>100 THEN x-10 ELSE 91
51
52 Qy1 == BF(y1)
53 Qrte == BF(y1) /\ BF(y2) /\ BF(z)
54 Question == Qpartialcorrectness /\ Qrte
55
56 QQ == 0 \leq y2 /\ y2 \leq 2
57
58 test == QQ
59
60 =====

```

Listing 2 – Configuration TLC

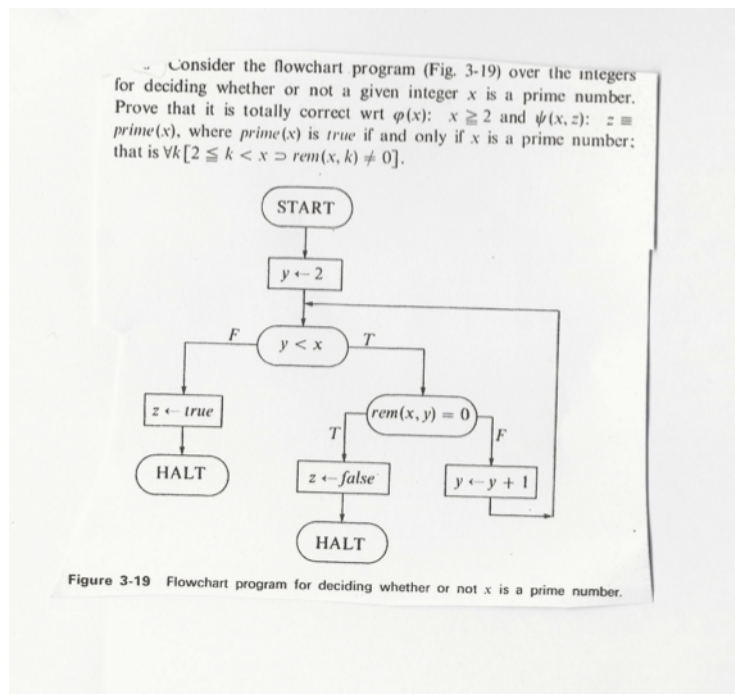
```

1  \* SPECIFICATION
2  \* Uncomment the previous line and provide the specification name if it's
   declared
3  \* in the specification file. Comment INIT / NEXT parameters if you use
   SPECIFICATION.
4
5  CONSTANTS
6      greeting = "Hello"
7      max = 1000
8      u = 1001
9      x = 101
10
11  INIT Init
12  NEXT Next
13
14  \* PROPERTY
15  \* Uncomment the previous line and add property names
16
17  INVARIANT
18  \* Uncomment the previous line and add invariant names
19  Question

```

Fin 1

Exercice 2 Soit le schéma suivant définissant un calcul déterminant si un nombre entier naturel est premier ou non.



Question 2.1 Ecrire un modèle TLA modélisant ce schéma de calcul.

Question 2.2 Ecrire un invariant à partir d'annotations que vous définirez après avoir défini des points de contrôle.

Question 2.3 Vérifier la correction partielle

Question 2.4 Vérifier l'absence d'erreurs à l'exécution.

◇— **Solution de l'exercice 2**

Listing 3 – Module TLA+

```

1  ----- MODULE appex2_3
2  -----
3  (* prime numbers *)
4  EXTENDS Naturals, TLC, Integers
5  CONSTANTS x0, max, u
6  min == -max
7  VARIABLES x, y, z, pc
8  D == min..max
9  rte(X) == X#u => X \in D
10 -----
11 (* PrÃ©condition *)
12 ASSUME x0 \in D /\ x0 \geq 2
13 -----
14 (* dÃ©finitions *)
15 diviseurs(X) == { m \in 1..X : X % m = 0 }
16 prime(X) == (diviseurs(X) = {1, X}) /\ X # 1
17 Locs == { "START", "HALT", "POINT" }
18 -----
19 start == pc="START" /\ y'=2 /\ pc'="POINT" /\ UNCHANGED <<x, z>>
20
21 case1 ==
22     /\ pc="POINT" /\ y \geq x
23     /\ z'=TRUE
24     /\ pc'="HALT"
25     /\ PrintT(y)
26     /\ UNCHANGED <<x, y>>
27
28 case21 ==
29     /\ pc="POINT" /\ y<x /\ (x % y = 0)
30     /\ pc'="HALT"
31     /\ z'=FALSE
32     /\ UNCHANGED <<x, y>>
33
34 case22 ==
35     /\ pc="POINT" /\ y<x /\ (x % y # 0)
36     /\ y'=y+1
37     /\ UNCHANGED <<x, z, pc>>
38
39 eprint ==
40     /\ pc="HALT"
41     /\ PrintT(z)
42     /\ PrintT(x)
43     /\ UNCHANGED <<x, y, z, pc>>
44
45 -----
46 Next ==
47     \/ start \/ case1 \/ case21 \/ case22
48     /\ UNCHANGED <<x, y, z, pc>> \/ eprint
49
50
51 Init == x=x0 /\ y = u /\ z = u /\ pc = "START"
52 -----
53
54
55 Q1 == pc # "HALT" (* pc prned la valeur HALT *)

```

```

56 Q2 == pc ="HALT" => (x=x0) /\ (z <=> (diviseurs(x)={1,x} /\ x#1))
57 Q3 == pc ="HALT" => (x=x0) /\ (z = prime(x))
58 Q4 == pc \in Locs
59 Q5 == rte(x) /\ rte(y)
60
61 Q == Q2 /\ Q3 /\ Q4 /\ Q5
62
63
64 =====

```

Listing 4 – Configuration TLC

```

1 \* SPECIFICATION
2 \* Uncomment the previous line and provide the specification name if it's
   declared
3 \* in the specification file. Comment INIT / NEXT parameters if you use
   SPECIFICATION.
4
5 CONSTANTS
6     greeting = "Hello"
7     x0 = 23
8     max = 1000
9     u = 100000
10
11 INIT Init
12 NEXT Next
13
14 \* PROPERTY
15 \* Uncomment the previous line and add property names
16
17 \* INVARIANT
18 \* Uncomment the previous line and add invariant names
19 CHECK_DEADLOCK TRUE

```

Fin 2

Exercice 3 *Le module truc permet de résoudre un problème très classique en informatique : trouver un chemin entre un sommet input et des sommets output supposés être des sommets de sortie.*

Question 3.1 *Pour trouver un chemin de input à l'un des sommets de output, il faut poser une question de sûreté à notre système de vérification. Donner une question de sûreté à poser permettant de trouver un chemin de input vers un sommet de output.*

Question 3.2 *On désire utiliser cette technique pour trouver un chemin dans un labyrinthe. Un labyrinthe est représenté par une matrice carrée de taille n . On définit ensuite pour chaque élément $\langle\langle i, j \rangle\rangle$ de la matrice les voisins communiquant à l'aide de la fonction lab qui associe à $\langle\langle i, j \rangle\rangle$ les éléments qui peuvent être atteints en un coup. Par exemple, le mouvement possible à partir de $\langle\langle 1, 1 \rangle\rangle$ est $\langle\langle 2, 1 \rangle\rangle$, ou le mouvement possible à partir de $\langle\langle 2, 1 \rangle\rangle$ est $\langle\langle 2, 2 \rangle\rangle$ ou $\langle\langle 1, 1 \rangle\rangle$, ou le mouvement possible à partir de $\langle\langle 2, 2 \rangle\rangle$ est $\langle\langle 2, 3 \rangle\rangle$ ou $\langle\langle 3, 2 \rangle\rangle$ ou $\langle\langle 2, 1 \rangle\rangle$, ...*

```

lab == [<<x,y>> \in (nodes \X nodes) |->
        IF x=1 /\ y=1 THEN {<<2,1>>} ELSE
        IF x=2 /\ y=1 THEN {<<2,2>>}
        IF x=1 /\ y=2 THEN {} ELSE
        IF x=2 /\ y=2 THEN {<<3,2>>,<<2,3>>} ELSE
        ELSE {}
    ]

```

MODULE *Naturals*

A dummy module that defines the operators that are defined by the real *Naturals* module.

$Nat \triangleq \{ \}$
 $a+b \triangleq \{a, b\}$
 $a-b \triangleq \{a, b\}$
 $a \cdot b \triangleq \{a, b\}$
 $a^b \triangleq \{a, b\}$
 $a < b \triangleq a = b$
 $a > b \triangleq a = b$
 $a \leq b \triangleq a = b$
 $a \geq b \triangleq a = b$
 $a \% b \triangleq \{a, b\}$
 $a \div b \triangleq \{a, b\}$
 $a .. b \triangleq \{a, b\}$

MODULE *TLC*

LOCAL INSTANCE *Naturals*
 LOCAL INSTANCE *Sequences*

$Print(out, val) \triangleq val$
 $PrintT(out) \triangleq TRUE$
 $Assert(val, out) \triangleq \text{IF } val = TRUE \text{ THEN } TRUE$
 $ELSE \text{ CHOOSE } v : TRUE$
 $JavaTime \triangleq \text{CHOOSE } n : n \in Nat$
 $TLCGet(i) \triangleq \text{CHOOSE } n : TRUE$
 $TLCSet(i, v) \triangleq TRUE$

$d \rightarrow e \triangleq [x \in \{d\} \mapsto e]$
 $f @@@ g \triangleq [x \in (\text{DOMAIN } f) \cup (\text{DOMAIN } g) \mapsto$
 $\text{IF } x \in \text{DOMAIN } f \text{ THEN } f[x] \text{ ELSE } g[x]]$
 $Permutations(S) \triangleq$
 $\{f \in [S \rightarrow S] : \forall w \in S : \exists v \in S : f[v] = w\}$

In the following definition, we use *Op* as the formal parameter rather than *\prec* because TLC Version 1 can't handle infix formal parameters.

$SortSeq(s, Op(_, _)) \triangleq$
 $LET \text{Perm} \triangleq \text{CHOOSE } p \in Permutations(1 .. Len(s)) :$
 $\forall i, j \in 1..Len(s) :$
 $(i < j) \Rightarrow Op(s[p[i]], s[p[j]]) \vee (s[p[i]] = s[p[j]])$
 $IN [i \in 1..Len(s) \mapsto s[Perm[i]]]$

$RandomElement(s) \triangleq \text{CHOOSE } x \in s : TRUE$

$Any \triangleq \text{CHOOSE } x : TRUE$

$ToString(v) \triangleq (\text{CHOOSE } x \in [a : v, b : STRING] : TRUE).b$

$TLCEval(v) \triangleq v$

FIGURE 1 – Modules *Naturals.tla* et *TLC.tla*

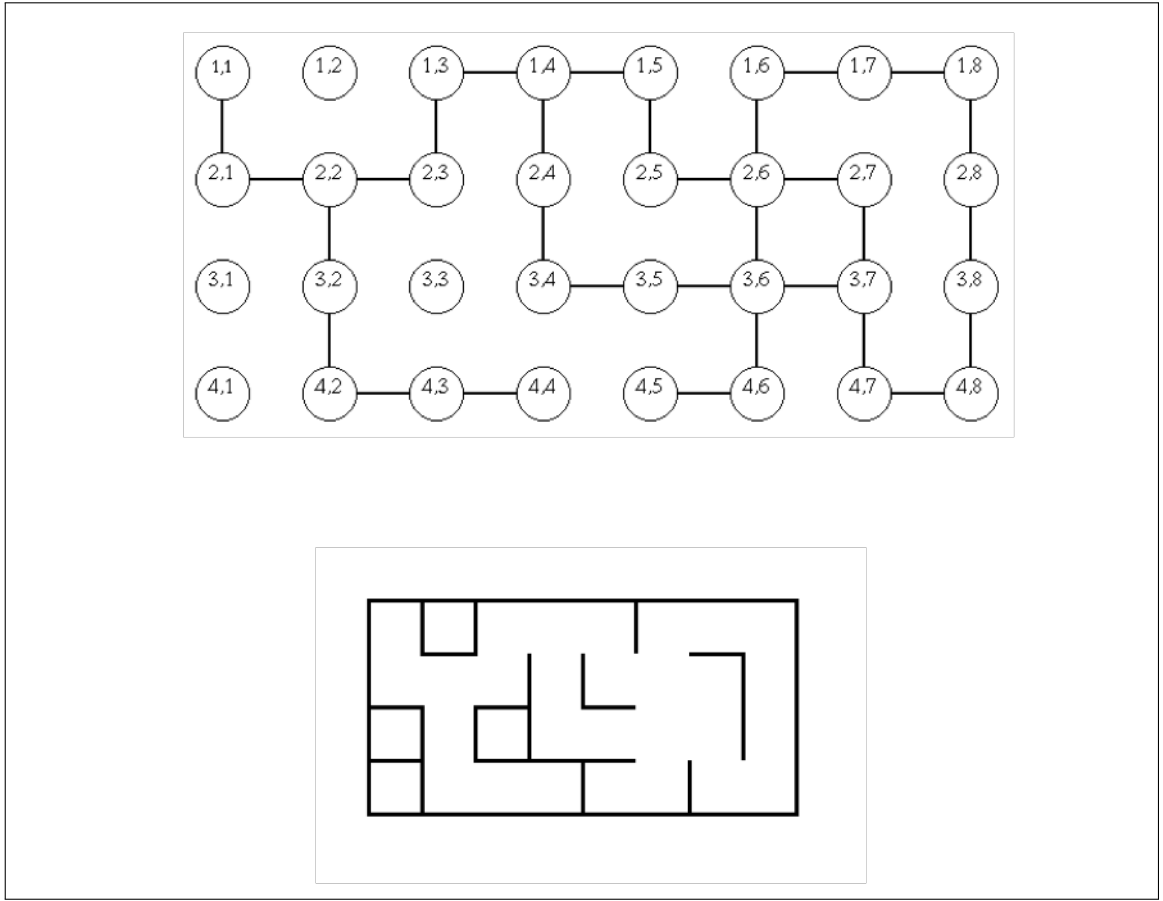


FIGURE 2 – Labyrinthe

Modifier le module truc pour traiter ce problème et donner la question à poser pour trouver une sortie.

```

MODULE truc
EXTENDS Integers, TLC
VARIABLES p
CONSTANTS input, output

n ≜ 10
nodes ≜ 1..n
l ≜ [i ∈ 1..n ↦ IF i = 1 THEN {4, 5} ELSE
  IF i = 2 THEN {6, 7, 10} ELSE
  IF i = 4 THEN {7, 8} ELSE
  IF i = 5 THEN {} ELSE
  IF i = 6 THEN {4} ELSE
  IF i = 7 THEN {5} ELSE
  IF i = 8 THEN {5, 2} ELSE
  {}
]

Init ≜ p = 1
M(i) ≜ ∧ i ∈ l[p]
      ∧ p' = i

```

$Next \triangleq \exists i \in 1..n : M(i)$

◊ Solution de l'exercice 3

Listing 5 – Module TLA+

```

1  ----- MODULE appex2_4
2  -----
3  EXTENDS Integers, TLC
4  VARIABLES p
5  CONSTANTS input, output
6
7  n == 10
8  nodes == 1..n
9  l == [i \in 1..n |-> IF i=1 THEN {4,5} ELSE
10                      IF i=2 THEN {6,7,10} ELSE
11                      IF i=4 THEN {7,8} ELSE
12                      IF i=5 THEN {} ELSE
13                      IF i=6 THEN {4} ELSE
14                      IF i = 7 THEN {5} ELSE
15                      IF i = 8 THEN {5, 2} ELSE
16                      {}
17                      ]
18
19  lab == [<<x,y>> \in (nodes \X nodes) |->
20          IF x=1 /\ y=1 THEN {<<1,2>>} ELSE
21          IF x=1 /\ y=2 THEN {<<1,1>>, <<1,3>>, <<2,2>>} ELSE
22          IF x=1 /\ y=3 THEN {<<1,2>>} ELSE
23          IF x=2 /\ y=2 THEN {<<1,2>>}
24          ELSE {}
25          ]
26
27  Init == p = 1
28  M(i) == /\ i \in l[p]
29          /\ p'=i
30  Next == \E i \in 1..n: M(i)
31
32  Initlab == p = input
33  ML(q) == /\ q \in lab[p]
34           /\ p'=q
35  Nextlab == \E q \in nodes \X nodes : ML(q)
36
37  Sortie == p \notin output
38
39  =====
40  =====

```

Listing 6 – Configuration TLC

```

1  \* SPECIFICATION
2  \* Uncomment the previous line and provide the specification name if it's
   declared
3  \* in the specification file. Comment INIT / NEXT parameters if you use
   SPECIFICATION.
4
5  CONSTANTS
6      greeting = "Hello"
7      input = {1}

```

```
8      output = {5}
9
10  INIT Init
11  NEXT Next
12
13  \* PROPERTY
14  \* Uncomment the previous line and add property names
15
16  INVARIANT
17  \* Uncomment the previous line and add invariant names
18  Sortie
19  CHECK_DEADLOCK FALSE
```

Fin 3